Project Proposal

Parallel 2D Heat Diffusion Solver using OpenMP

1. Introduction

Heat diffusion is a fundamental physical process describing how thermal energy spreads within a medium over time. It is governed by the heat equation, a partial differential equation (PDE) widely used in physics, engineering, and computational sciences.

In this project, we aim to develop a **numerical solver for the two-dimensional heat diffusion equation** and accelerate its computation using **OpenMP-based parallelization**. The solver will simulate the evolution of temperature distribution over a rectangular domain and analyse performance improvements achieved through parallel computing.

---

2. Problem Statement

The objective of this project is to:

- Implement a **2D heat diffusion solver** using finite difference methods,
- Parallelize the computation using **OpenMP**,
- Analyze performance in terms of **execution time, speedup, and scalability**,
- Compare serial and parallel implementations.

The problem involves solving the time-dependent heat equation over a discretized spatial grid.

---

3. Mathematical Model

The governing equation for 2D heat diffusion is:

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right),$$

where:

- $T(x, y, t)$ is the temperature field,
- α is the thermal diffusivity constant.

Using finite difference discretization, the update equation becomes:

$$T_{i,j}^{n+1} = T_{i,j}^{n} + \alpha \Delta t \left(\frac{T_{i+1,j}^{n} - 2T_{i,j}^{n} + T_{i-1,j}^{n}}{\Delta x^2} + \frac{T_{i,j+1}^{n} - 2T_{i,j}^{n} + T_{i,j-1}^{n}}{\Delta y^2} \right).$$

---

4. Motivation for Choosing This Problem

This problem is selected based on the following considerations:

4.1 Suitability for Parallelization

- The computational domain is a **2D grid**, where each cell update depends only on neighboring values from the previous timestep.
- This leads to **data independence within each timestep**, making it highly suitable for parallel execution.
- Unlike particle-based simulations (e.g., DEM), **no race conditions occur**, simplifying parallel implementation.

4.2 Compatibility with OpenMP

- OpenMP is designed for **loop-level parallelism**, which aligns perfectly with grid-based updates.
- Nested loops over grid indices can be parallelized using:
  - #pragma omp parallel for
  - collapse(2) for 2D loops

4.3 Computational Characteristics

- The algorithm scales as:

$$\mathcal{O}(N \times M \times T),$$

where $N \times M$ is the grid size and T is the number of timesteps.

- Large grids provide **sufficient workload per thread**, ensuring good parallel efficiency.

4.4 Visualization and Interpretability

- The results can be visualized as **heatmaps**, making it easy to interpret temperature evolution.
- This improves both **presentation quality** and conceptual understanding.

---

5. Numerical Algorithm

The simulation follows a time-stepping approach:

1. Initialize temperature grid with boundary and initial conditions.
2. For each timestep:
   - Compute temperature updates for all interior grid points,
   - Apply boundary conditions,
   - Swap grids for next iteration.

---

6. Parallelization Strategy

The primary computation involves updating each grid cell independently:

$$T_{i,j}^{n+1} \leftarrow f(T_{i,j}^{n}, \text{neighbors})$$

Since all updates depend only on the previous timestep, they can be executed concurrently.

8. Expected Outcomes

- Significant reduction in runtime with increasing threads
- Near-linear speedup for moderate thread counts
- High efficiency due to minimal synchronization overhead

---

9. Conclusion

This project demonstrates how a classical physics problem can be efficiently solved using parallel computing techniques. The 2D heat diffusion solver provides an ideal platform to study OpenMP-based parallelization due to its structured computation, absence of race conditions, and scalability.

The insights gained from this project will contribute to a deeper understanding of numerical methods, parallel programming, and performance optimization.